

Programming AI Accelerators

Course Project Guidelines

1. Project Objective

The course project provides hands-on experience in **programming, optimizing, and evaluating AI workloads on modern computing accelerators**.

Projects should involve one or more of:

- GPU/accelerator programming and CUDA
- AI inference/training optimization
- LLMs and foundation models
- Kernel optimization and code generation
- Model compression and quantization
- Multi-GPU systems
- Hardware-aware AI
- Edge AI
- Performance, power, and energy efficiency

The emphasis is on **implementation, measurement, profiling, optimization, and technical analysis**.

Measure first. Understand the bottleneck. Optimize second.

2. Project Catalogue

Students may select one of the following broad topics and convert it into a specific technical problem.

Energy & Performance Benchmarking

Performance–Power Characterization of AI Workloads

Benchmark AI workloads across different models, batch sizes, precisions, and GPU configurations. Study the trade-off between **latency, throughput, power, and energy/inference**.

Energy-Efficient AI Kernel Optimization

Implement and optimize kernels such as GEMM, attention, convolution, or reduction. Study

whether techniques such as **tiling, fusion, memory optimization, and low precision** improve both performance and energy efficiency.

Energy-Efficient LLM Inference

Study the relationship between **tokens/sec, latency, GPU memory, power, and energy/token** for LLM inference. Possible variables include batch size, context length, quantization, KV-cache optimization, and number of GPUs.

Multi-GPU Performance and Energy Scaling

Study how AI workloads scale from **1 → 2 → 4+ GPUs**, considering computation, communication, power, and energy. Investigate tensor/pipeline parallelism and communication overhead, with the goal of identifying the **best performance–energy operating point**.

GPU & Kernel Systems

GPU Kernel Optimization for AI Workloads

Design and optimize CUDA kernels for operations such as GEMM, attention, convolution, reduction, normalization, and other AI primitives. Explore memory access, tiling, occupancy, parallelism, precision, and architecture-specific optimizations. NVIDIA's current best-practice guidance emphasizes parallelism, memory efficiency, and instruction efficiency as the core optimization dimensions.

Kernel Fusion & MegaKernels for AI Inference

Investigate combining multiple AI operations into larger fused kernels or **mega-kernels** to reduce kernel-launch overhead, intermediate memory traffic, and synchronization. Possible targets include LLMs, VLMs, diffusion models, and other foundation models.

GPU Kernel Generation & Auto-Tuning

Explore automatic generation, specialization, and tuning of GPU kernels. Students could investigate CUDA, Triton, CUTLASS, compiler-generated kernels, or search-based approaches to find configurations optimized for a particular workload and GPU architecture. Current CUTLASS tooling, for example, supports both optimized C++ and Python-generated kernels across NVIDIA architectures including Blackwell.

GPU Performance Analysis & Architecture-Aware Optimization

Benchmark and profile workloads to understand **why** they perform the way they do. Explore memory-bound vs. compute-bound behaviour, occupancy, instruction throughput, memory hierarchy, warp behaviour, Tensor Cores, and architecture-specific features. Projects could compare the same kernel across different GPU generations, including Blackwell.

Efficient & Specialized GPU Kernels

Focus on kernels for challenging or specialized workloads such as **sparse computation, low-precision computation, irregular workloads, small-batch inference, or low-end GPUs**. The goal could be to design kernels that outperform general-purpose implementations under specific workload or hardware constraints.

LLMs & Foundation Models

Efficient LLM Inference & Serving

Optimize LLM inference for latency, throughput, memory, and scalability. Possible directions include batching, scheduling, prefill/decode optimization, prefix caching, and efficient serving.

KV Cache Optimization for Long-Context Models

Explore KV-cache compression, quantization, memory management, paging, eviction, and efficient attention for long-context and high-concurrency inference. KV-cache management is a major memory bottleneck in autoregressive inference.

Quantization & Low-Precision Foundation Models

Investigate FP8, INT8, INT4, NVFP4, or other low-precision approaches for reducing memory footprint and improving inference efficiency while maintaining model quality.

Fast Generation: Speculative Decoding & Efficient Attention

Explore techniques that reduce the cost of autoregressive generation, including speculative decoding, FlashAttention-style approaches, attention variants, and other methods for accelerating token generation.

Efficient Foundation Models Beyond LLMs

Optimize inference for **VLMs, VLAs, diffusion models, and other foundation models**, with emphasis on kernels, memory, quantization, model architecture, and hardware-aware optimization.

Multi-GPU & Distributed AI

Multi-GPU AI Inference

Scaling LLMs and other foundation models across multiple GPUs using model/tensor/context parallelism.

Tensor & Pipeline Parallelism

Exploring different ways of partitioning models across GPUs and studying the trade-offs between memory, computation, communication, and latency.

Multi-GPU Communication & Collective Operations

Optimizing AllReduce, AllGather, ReduceScatter, All-to-All and related primitives using NCCL, CUDA P2P, NVLink/NVSwitch, etc.

Communication-Computation Overlap & Scaling

Hiding communication behind GPU computation, optimizing synchronization, and studying scaling efficiency as the number of GPUs increases.

Distributed & Heterogeneous AI Systems

Multi-node AI workloads, topology-aware execution, heterogeneous GPU configurations, and efficient deployment across different accelerator/interconnect configurations. NVIDIA's multi-GPU examples explicitly cover both single-node and cluster-based programming models.

Applications & Edge AI

Edge Computer Vision & Video Analytics

Real-time object detection, tracking, segmentation, anomaly detection, and intelligent video processing on edge GPUs/devices.

Vision-Language AI at the Edge

Efficient deployment of VLMs for visual assistants, scene understanding, video summarization, and natural-language interaction with cameras.

AI for Robotics & Autonomous Systems

Accelerating perception, planning, and Vision-Language-Action models for robots and other autonomous machines.

AI for Resource-Constrained Edge Devices

Deploying and optimizing AI on Jetson, embedded GPUs, microcontrollers, Coral-class devices, and other low-power platforms, with emphasis on latency, memory, and energy.

Domain-Specific Edge AI Applications

Applying accelerator programming to domains such as **healthcare, geospatial analysis, industrial inspection, IoT, smart infrastructure, and environmental monitoring**.

Students may also propose an **original project**. Student-proposed projects are encouraged if they have a strong connection to AI acceleration, efficient AI systems, accelerator programming, model efficiency, or related systems topics.

3. Project Proposal

The selected topic is only a starting point. Each team must turn it into a **specific, measurable technical problem**.

The proposal should briefly describe:

Problem Statement and Motivation

Proposed Approach

Hardware and Software Platform

Baseline

Evaluation Metrics

Expected Contribution

Risks / Fallback Plan

Project Timeline

A good proposal should clearly answer:

What are we trying to improve, why is it difficult, how will we improve it, and how will we measure success?

4. Project Methodology

Projects should generally follow:

Problem

↓

Baseline

↓

Benchmark

↓

Profile

↓

Identify Bottleneck

↓

Optimize

↓

Benchmark Again

↓

Analyze Results

Students should establish a **reliable baseline before optimization**.

Relevant metrics may include:

- Latency and throughput
- Memory usage and bandwidth
- GPU utilization
- Kernel execution time
- Communication overhead
- Scaling efficiency
- Power and energy
- Model accuracy/quality

For NVIDIA GPUs, students are encouraged to use tools such as **Nsight Compute, Nsight Systems, and NVIDIA SMI** for performance analysis.

A large speedup is not required. **Rigorous measurement and a clear explanation of results are more important than an impressive but unsupported number.**

5. Nine-Week Schedule

Week	Activity
1	Project selection, team formation & initial reading
2	Detailed proposal & scope finalization
3	Baseline implementation & benchmarking
4	Profiling & bottleneck analysis
5	Evaluation 1 — Baseline & Problem Understanding
6	Main implementation / optimization
7	Evaluation 2 — Working Prototype
8	Optimization, experiments & final benchmarking
9	Final Evaluation — Demo, Presentation & Report

6. Evaluations

Evaluation 1 — Baseline & Problem Understanding

Students should demonstrate:

- Clear understanding of the problem

- Working baseline
- Reliable measurements
- Initial profiling
- Identified bottleneck
- Proposed optimization strategy

Key question:

What exactly are we trying to improve, and how do we know it is the bottleneck?

Evaluation 2 — Working Prototype

Students should demonstrate:

- Working implementation
- Baseline comparison
- Preliminary results
- Profiling/analysis
- Feasibility of the proposed approach

Key question:

Does our proposed approach actually work?

Final Evaluation

Students should present:

- Complete implementation
- Final performance results
- Profiling analysis
- Important experiments/ablations
- Accuracy/quality analysis where applicable
- Limitations
- Working demonstration

7. Evaluation Weightage

Component	Weight
Project Proposal	10%
Evaluation 1 — Baseline & Problem Understanding	20%

Evaluation 2 — Working Prototype	25%
Final Implementation & Experiments	25%
Final Presentation / Demonstration	10%
Technical Report	10%
Total	100%

8. Team & Hardware Guidelines

Teams

Recommended team size: **2–3 students**.

Larger teams require sufficient project scope and justification. Each member must have a meaningful technical contribution.

Hardware

Projects may use platforms such as:

- NVIDIA GPUs / Blackwell
- Multi-GPU systems
- Jetson
- TPU
- Tenstorrent
- Cerebras
- Coral
- Other accelerators

Verify hardware availability before finalizing the project. If specialized hardware is required, include a fallback plan in the proposal.

9. What Makes a Strong Project?

Projects will be judged primarily on:

- Problem understanding
- Technical depth
- Implementation quality

Profiling and performance analysis
Experimental rigor
Optimization
Technical contribution
Ability to explain the results

The project does **not** have to produce a dramatic speedup.

A careful analysis explaining **why an optimization succeeds or fails** is a valuable outcome.

Final Principle

Don't just make it work. Understand the bottleneck, measure it carefully, and make it better.